MD. ABDUL AL MOHIT, (D.Eng)
Professor,
Department of Mathematics
Islamic University, Bangladesh
Former Researcher

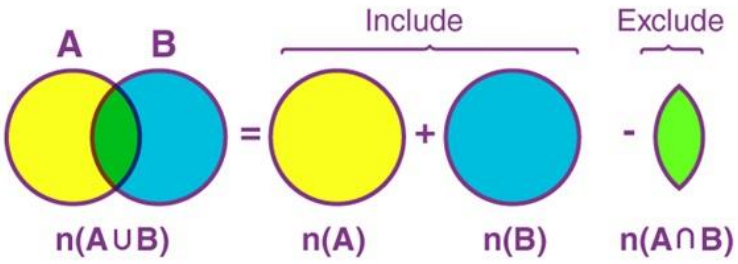
Disaster Risk Reduction Research Centre, Faculty of Engineering, Kyushu University, 744, Motooka, Nishiku, Fukuoka 819-0395, Japan.

1

Sum rule principle: If A and B are disjoint set then n(AUB)=n(A)+n(B).

Product rule principle: Let AXB be the Cartesian product of sets A and B then n(AXB)=n(A).n(B)

Principle of Inclusion and Exclusion is an approach which derives the method of finding the number of elements in the union of two finite sets. This is used to solve combinations and probability problems when it is necessary to find a counting method, which makes sure that an object is not counted twice.



If A and B are disjoint set then show that n(AUB)=n(A)+n(B).

A and B are two disjoint sets.
$\therefore A \cap B = \phi \Longrightarrow n(A \cap B) = 0$
Since, $n(A \cup B) = n(A) + n(B) - n(A \cap B)$
$= n(A) + n(B) - 0 = n(A) + n(B)$
Hence, $n(A \cup B) = n(A) + n(B)$

Theorem: For any two finite set A and B then show $n(A \cup B)=n(A)+n(B)-n(A \cap B)$

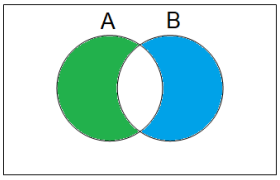
Proof: For any two dij joint set A and B, we have

$$n(A \cup B)=n(A)+n(B) \quad (1)$$

Here, A is the disjoint union of A-B and A∩B. B is the Disjoint Union of B-A and A∩B. Thus AUB is the disjoint union of A-B, A∩B and B-A.

Therefore, by equation (1) we have,

$$n(A)=n(A-B)+n(A \cap B)$$
$$n(B)=n(B-A)+n(A \cap B)$$



And

$$n(A \cup B)=n(A-B)+n(A \cap B)+n(B-A)$$
$$= n(A-B)+n(A \cap B)+n(B-A)+n(A \cap B)-n(A \cap B)$$
$$=n(A)+n(B)-n(A \cap B) \quad \text{(Proved)}$$

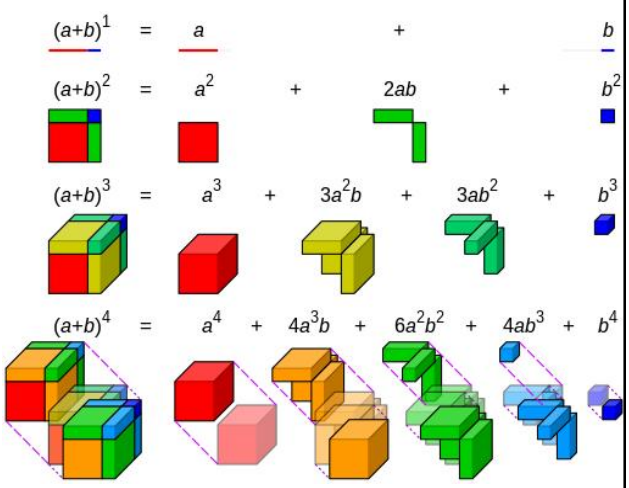
Binomial coefficients, positive integers that are the numerical coefficients of the binomial theorem, which expresses the expansion of $(a + b)^n$. The nth power of the sum of two numbers a and b may be expressed as the sum of $n + 1$ terms of the form

$$\binom{n}{r} a^{n-r} b^r;$$

in the sequence of terms, the index r takes on the successive values 0, 1, 2,…, n. The binomial coefficients are defined by the formula

$$\binom{n}{r} = n!/(n-r)!r!,$$

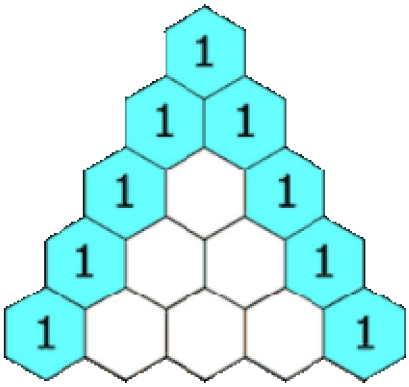
in which $n!$ (called n factorial) is the product of the first n natural numbers 1, 2, 3,…, n (and where 0! is defined as equal to 1). The coefficients may also be found in the array often called Pascal's triangle

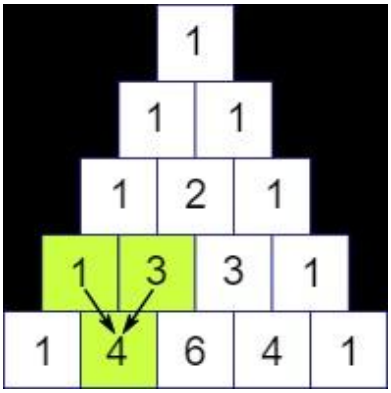


Pascal's triangle, in algebra, a triangular arrangement of numbers that gives the coefficients in the expansion of any binomial expression, such as $(x + y)^n$. It is named for the 17th-century French mathematician Blaise Pascal, but it is far older.

Pascal's Triangle is a kind of number pattern. Pascal's Triangle is the triangular arrangement of numbers that gives the coefficients in the expansion of any binomial expression. The numbers are so arranged that they reflect as a triangle. Firstly, 1 is placed at the top, and then we start putting the numbers in a triangular pattern. The numbers which we get in each step are the addition of the above two numbers. It is similar to the concept of triangular numbers.

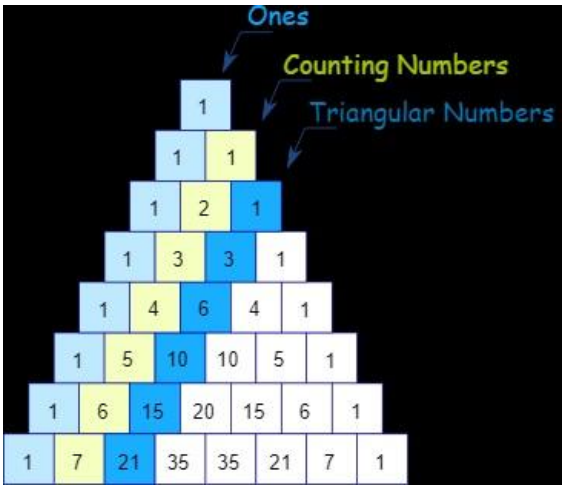
One of the most interesting Number Patterns is Pascal's Triangle





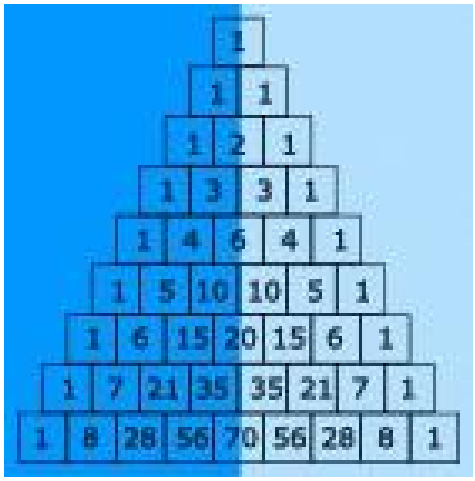
Patterns Within the Triangle

Diagonals



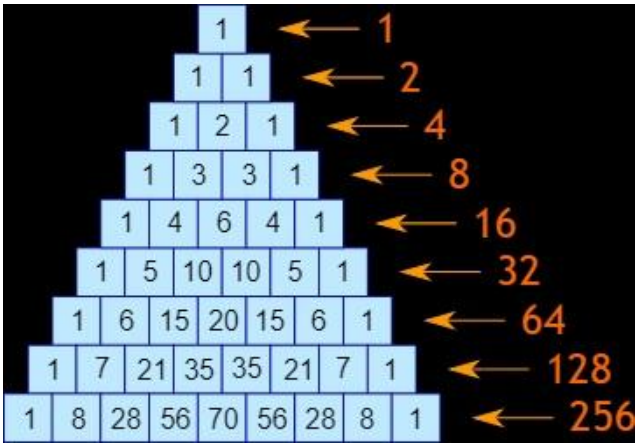
Symmetrical

The triangle is also symmetrical. The numbers on the left side have identical matching numbers on the right side, like a mirror image.



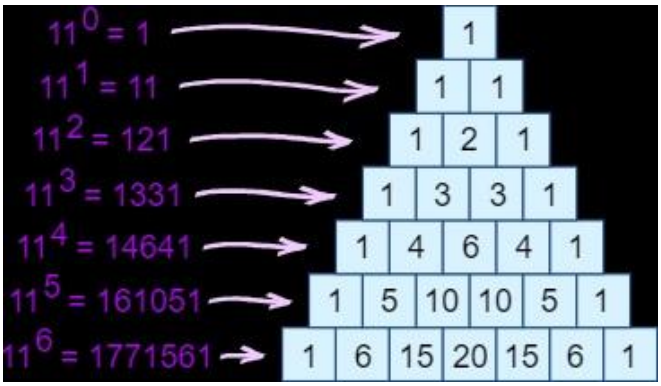
Horizontal Sums

What do you notice about the horizontal sums?
Is there a pattern? They **double** each time



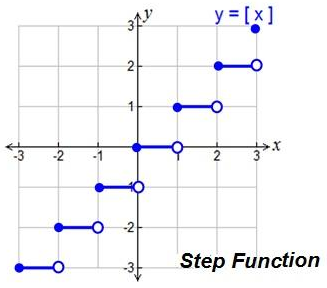
Exponents of 11
Each line is also the powers (exponents) of 11:

110=1 (the first line is just a "1")
111=11 (the second line is "1" and "1")
112=121 (the third line is "1", "2", "1")



Step function

In Mathematics, a step function (also called as staircase function) is defined as a piecewise constant function, that has only a finite number of pieces. In other words, a function on the real numbers can be described as a finite linear combination of indicator functions of given intervals



A step function f: R → R can be written in the form:

$$f(x) = \sum_{i=0}^{n} \alpha_i \chi_{A_i}(x)$$

for all real numbers x. If $n \geq 0$, α_i are real numbers and A_i are intervals, then the indicator function of A is χ_A, and it can be written as below:

$$\chi_A(x) = \begin{cases} 1; if\ x \in A, \\ 0; if\ x \notin A \end{cases}$$

$$f(x) = \begin{cases} 1, & 0 \leq x < 2 \\ 3, & 2 \leq x < 4 \\ -1, & 4 \leq x < 6 \end{cases}$$

Floor function

In mathematics and computer science, the **floor function** is the function that takes as input a real number x, and gives as output the greatest integer less than or equal to x, denoted $\lfloor x \rfloor$ or floor(x).

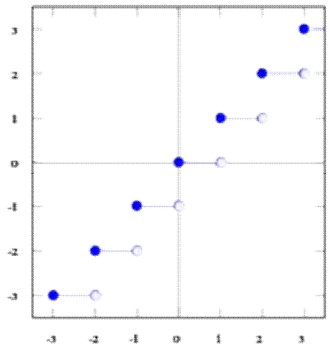
The floor and ceiling functions give us the nearest integer up or down.

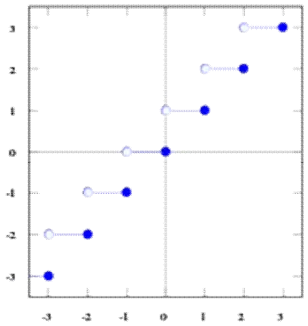
The floor function always rounds down to the nearest whole number.

Input x	Floor ⌊x⌋	Explanation
7.9	7	7 is the largest integer ≤ 7.9
5.0	5	5 is already an integer
2.3	2	2 is the largest integer ≤ 2.3
0.99	0	0 is the largest integer ≤ 0.99
-1.2	-2	-2 is the largest integer ≤ -1.2
-4.0	-4	-4 is already an integer
-6.8	-7	-7 is the largest integer ≤ -6.8

Ceiling function

In mathematics and computer science, the **ceiling function** maps x to the least integer greater than or equal to x, denoted $\lceil x \rceil$ or ceil(x).





Example: What is the floor and ceiling of 2.31?

The Floor of 2.31 is **2**

The Ceiling of 2.31 is **3**

Floor

$$f(x) = \lfloor x \rfloor = \begin{cases} \vdots & \vdots \\ -2 & \text{if } -2 \le x < -1 \\ -1 & \text{if } -1 \le x < 0 \\ 0 & \text{if } \ 0 \le x < 1 \\ 1 & \text{if } \ 1 \le x < 2 \\ 2 & \text{if } \ 2 \le x < 3 \\ \vdots & \vdots \end{cases}$$

Ceiling

$$f(x) = \lceil x \rceil = \begin{cases} \vdots & \vdots \\ -1 & \text{if } -2 < x \le -1 \\ 0 & \text{if } -1 < x \le 0 \\ 1 & \text{if } \ 0 < x \le 1 \\ 2 & \text{if } \ 1 < x \le 2 \\ 3 & \text{if } \ 2 < x \le 3 \\ \vdots & \vdots \end{cases}$$

Simple Algorithm

Step 1: Start
Step 2: Get the number * Let the given decimal number be x.
Step 3: Find the Floor value (Round Down)
•Drop the decimal part of x to get a whole number. Let's call it temp.
•If x has no decimals (it is already a whole number), then Floor = temp.
•If x is negative (like -5.1), then Floor = temp minus 1 (becomes -6).
•Otherwise, for positive numbers, Floor = temp.
Step 4: Find the Ceiling value (Round Up)
•Drop the decimal part of x to get a whole number. Let's call it temp.
•If x has no decimals, then Ceiling = temp.
•If x is positive (like 5.9), then Ceiling = temp plus 1 (becomes 6).
•Otherwise, for negative numbers, Ceiling = temp.
Step 5: Print the answers
•Show the calculated Floor and Ceiling values.
Step 6: End

```
#include <stdio.h>
#include<math.h>

int main() {
   double x;
   int floor_val, ceiling_val;

   // Step 2: Get the number
   printf("Enter a decimal number: ");
   if (scanf("%lf", &x) != 1)
      {
      printf("Invalid input.\n");
      return 1;
   }
```

```
// Step 3: Find the Floor value (Round Down)
   int temp_floor = (int)x; // Drop the decimal part of x

   if (x == temp_floor)
      {
      floor_val = temp_floor; // If x has no decimals
     }
   else if (x < 0)
      {
      floor_val = temp_floor - 1; // If x is negative
   }
   else
      {
      floor_val = temp_floor; // Otherwise, for positive numbers
   }
```

```
// Step 4: Find the Ceiling value (Round Up)
   int temp_ceil = (int)x; // Drop the decimal part of x

   if (x == temp_ceil) {
      ceiling_val = temp_ceil; // If x has no decimals
   } else if (x > 0) {
      ceiling_val = temp_ceil + 1; // If x is positive
   } else {
      ceiling_val = temp_ceil; // Otherwise, for negative numbers
   }
```

```
// Step 5: Print the answers
    printf("\n--- Results ---\n");
    printf("Original Value: %.3f\n", x);
    printf("Floor Value:    %d\n", floor_val);
    printf("Ceiling Value:  %d\n", ceiling_val);

    return 0; // Step 6: End
}
```

The Algorithm of GCD

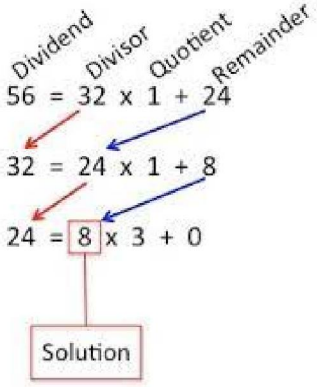
The Euclidean Algorithm for finding GCD(A,B) is as follows:

- If A = 0 then GCD(A,B)=B, since the GCD(0,B)=B, and we can stop.
- If B = 0 then GCD(A,B)=A, since the GCD(A,0)=A, and we can stop.
- Write A in quotient remainder form (A = B·Q + R)
- Find GCD(B,R) using the Euclidean Algorithm since GCD(A,B) = GCD(B,R)

Find the GCD of 270 and 192
A=270, B=192
A ≠0
B ≠0
Use long division to find that 270/192 = 1 with a remainder of 78. We can write this as: 270 = 192 * 1 +78
Find GCD(192,78), since GCD(270,192)=GCD(192,78)
A=192, B=78
A ≠0
B ≠0
Use long division to find that 192/78 = 2 with a remainder of 36. We can write this as:
192 = 78 * 2 + 36
Find GCD(78,36), since GCD(192,78)=GCD(78,36)

gcd(807, 481) = 1

807 = 481 x 1 + 326
481 = 326 x 1 + 155
326 = 155 x 2 + 16
155 = 16 x 9 + 11
16 = 11 x 1 + 5
11 = 5 x 2 + 1
5 = 1 x 5



```
#include <stdio.h>
#include<math.h>
int main() {
   int A, B;
   int original_A, original_B;

   // Get input values from user
   printf("Enter two integers (A and B): ");
   if (scanf("%d %d", &A, &B) != 2)
      {
      printf("Invalid input.\n");
      return 1;
   }
```

```
// Keep track of original values for the final print statement
   original_A = A;
   original_B = B;

   // Loop to apply Euclidean Algorithm rules continuously
   while (1)
      {
      // Rule 1: If A = 0 then GCD(A,B) = B, and we can stop.
      if (A == 0)
         {
         printf("\nGCD(%d, %d) = %d\n", original_A, original_B, B);
         break;
      }
```

```
// Rule 2: If B = 0 then GCD(A,B) = A, and we can stop.
      if (B == 0)
         {
         printf("\nGCD(%d, %d) = %d\n", original_A, original_B, A);
         break;
      }

      // Rule 3 & 4: Write A in quotient remainder form (A = B*Q + R)
      // In C, the remainder 'R' is found using the modulo operator '%'
      int R = A % B;

      // Find GCD(B,R) by shifting variables: B becomes the new A, R
becomes the new B
      A = B;
      B = R;
   }
   return 0;
}
```

Data sorting

Data sorting is any process that involves arranging the data into some meaningful order to make it easier to understand, analyze or visualize.

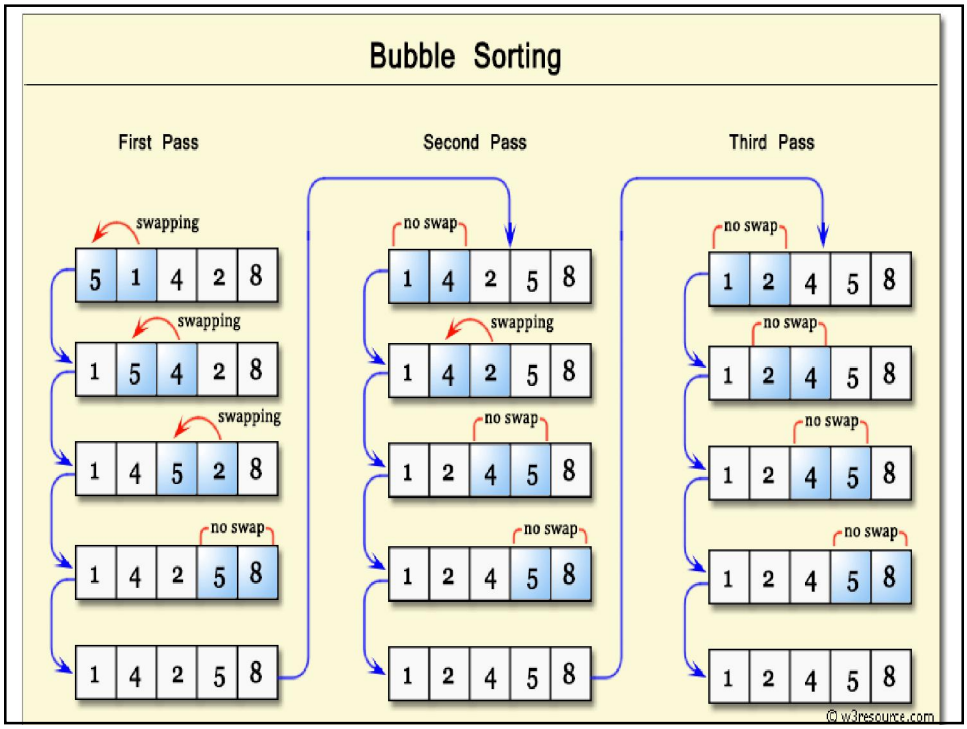
Bubble Sort

Bubble Sort is a simple algorithm which is used to sort a given set of n elements provided in form of an array with n number of elements. Bubble Sort compares all the element one by one and sort them based on their values.

Bubble Sort Algorithm

Following are the steps involved in bubble sort(for sorting a given array in ascending order):

1. Starting with the first element(index = 0), compare the current element with the next element of the array.
2. If the current element is greater than the next element of the array, swap them.
3. If the current element is less than the next element, move to the next element. **Repeat Step 1**.




```
for(i = 0; i < n; i++)
   {
      for(j = 0; j < n-i-1; j++)
      {
         if( arr[j] > arr[j+1])
         {
            // swap the elements
            temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
         }
      }
   }
```